

= PROBLEMA 1 =

Dacă a, b, c - lungimile a 3 lăţiri,
acestea vor forma un triunghi (eventual degenerat) dacă

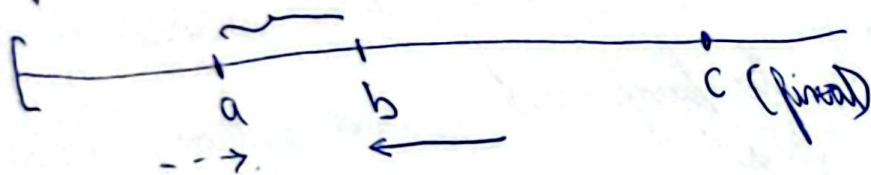
$$a+b \geq c, \quad a+c \geq b, \quad b+c \geq a$$

(dacă a, b, c - numere pozitive, e suficient să verificăm dacă $a+b \geq c$).

Notăm vectorul creşterii.

Pentru a număra triunghiurile valide (a, b, c) , vom folosi metoda two-pointers.

Fixăm cea mai mare valoare (c) care va parcurge vectorul de la dreapta la stânga. După această fixare, notăm la început a la poz. 0 și b la $(\text{poz}(c)-1)$ și vedem cum e numărul lor raportat la c .



unde $\text{poz}(a) < \text{poz}(b)$:

dacă $a+b \geq c \Rightarrow$ adăugăm $\text{poz}(b) - \text{poz}(a)$ la rezultat
(numărul va fi valid dacă a va fi în acel interval)
și mutăm b la stânga cu o poziție

dacă $a+b < c \Rightarrow$ mutăm a mai la dreapta.

→ după ce se închide bucla, reștim cu c mutat mai la stânga

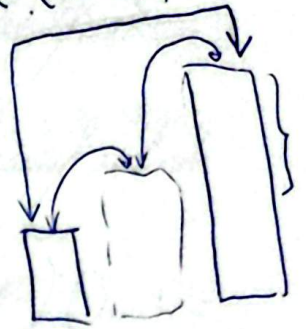
parcurgere two-pointers pt. fixat $\Rightarrow O(N)$ / $\Rightarrow O(N^2)$ - complexitate timp
poziții pe care le poate avea $c \Rightarrow O(N)$ / $O(N)$ - complexitate spațiu

= PROBLEMA 2 =

Am folosit căutare binară pentru a găsi diferența maximă (d_{max})

Căutare binară:

1) când eliminăm un element, diferența poate doar să crească (pentru că vectorul e sortat crescător)



2) e mai ușor să "bucăm/păstrăm" elemente decât să le mutăm
 $\Rightarrow$ avem de păstrat $N-K$ elemente.

Validare pentru căutare binară: dacă putem păstra cel puțin $N-K$ elemente din vîrul original a.i. diferența maximă nu depășește d_{max}
 (din nou, dacă adăugăm elemente, diferențele dintre ele pot doar să scadă)

Pentru căutare binară, limita de sus va fi $V[N-1] - V[0] + 1$.

Pentru a verifica dacă un d_{max} fixat este valid, facem un programare dinamică:

unde $dp[i]$ - numărul maxim de elemente (inclusiv $V[i]$!!) din vîrul $[0, i]$ astfel încît diferența dintre ele să fie maxim d_{max}

(hîm numărul maxim pentru că dacă avem mai multe decât ne trebuie, putem să selectăm o subsecvență continuă dintre acestea și nu vor avea diferența mai mare decât d_{max})

$$dp[i] = 1 + \max(dp[j]), \text{ unde } V[i] - V[j] \leq d_{max} \text{ (} i, j < i, \text{ evident)}$$

Pentru a găsi j -ul cel mai bine: știm că vectorul e sortat crescător $\Rightarrow$ diferențele doar vor scădea $\Rightarrow$ putem ține un indice id_x_j_min cu val. minimă pe care poate să o aibă ~~un~~ j a.i. $V[i] - V[j]$; acest id_x_j_min se va muta doar la dreapta și verificăm de acolo.

Dacă $\max(dp[i]) \text{ } i=1..n \geq N-K \Rightarrow d_{max}$ e valid



Notăm M - maximul din vector

$\Rightarrow$ complexitate citirea liniei: $O(\log M)$

Pentru fiecare verificare: avem parcurgerea pt. dp. $\Rightarrow O(N)$

$\Rightarrow O(N \log M)$ - complexitate timp

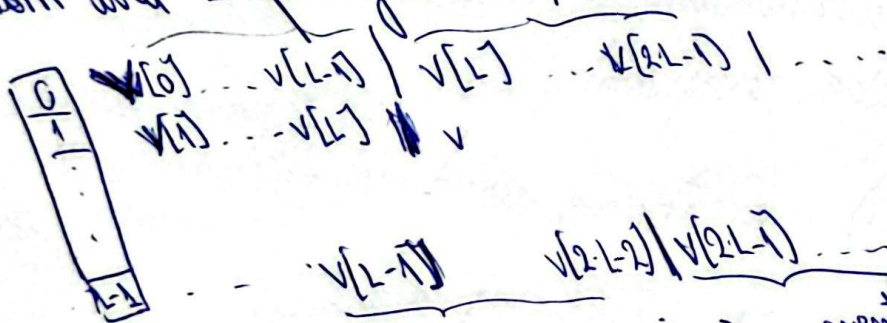
$O(N)$ - complexitate timp

= PROBLEMA 3 =

Neam să aflăm printre - o ringuire parcurgere simplă a unei bucăți din S de lungime L dacă aceasta este, de fapt, un curent din V (și care dintre acestea este).

Înțeles, dacă notăm cu K - nr. de curente distincte din V , vom construi un trie cu toate curentele distincte din V și vom asigna fiecărui indice de la 0 la $K-1$.
(am ține minte și frecvența curentelor / indicilor în vectorul v original).

Am avea L parcurgeri de nivelul S , câte una asociată unui rest modulo L .



Pentru că toate curentele au lungime L , eventualele ipoteze ~~ne~~ valide curentele concatenate sunt subsecvențe continue de blocuri în una dintre aceste L împărțiri după resturi.

~~Pentru un rest, care este~~
Pentru nivel dat unui rest (posibil puțin diferit), ne întoarcem la cele S/L blocuri și verificăm pentru fiecare dacă sunt reprezentative pentru un curent și, dacă da, care (parcurgând trie-ul).

Înțeles, fiecare bucăți va fi asociată unei valori care e ori indicele cur., ori -1
Bucăți $[i] = \begin{cases} \text{ide} \\ -1 \end{cases}$, unde $i \in \{0, \dots, S/L-1\}$
↳ nr. de bucăți.

Problema a fost redusă la a găsi succesiunea maximă de caractere concatenate
(unde unui caracter îi este asociat indicele lui) în vectorul lucrată.

Putem face asta folosind sliding window.

Dim. prezentele caract. fc ale fiec. caracter în vectorul v.

Când parcurgem vectorul lucrată, construim și actualizăm prezentele

de la o fereastră de lungime $\leq N$.

Verificăm în $O(1)$ dacă prezentele se potrivește cu cele fc folosind variabila `matches`. (verificăm când scadem sau creștem prezentele din capete cu 1 la rând dacă ele noi se potrivește sau nu și ~~actualizăm~~ actualizăm `matches` corespunzător)

Dim. că am găsit o concatenare când `matches = K`.

Repetăm acest proces pentru fiecare rest.

Dacă S - lungimea restului.

L - parcurgere string, verificăm în $O(1)$

$\Rightarrow O(S \cdot L)$ - complexitate timp

$O(N \cdot L)$ - complexitate spațiu (nr, vector, Trie, vectori de pres.)

= PROBLEMA 4 =

În primul rând, vom sorta după S_i taskurile.

Pentru că vom găsi cel mai mic indice disponibil, vom folosi un Heap care să țină toate valorile indicilor proceselor disponibile.

Când ~~se termină~~ apare un task, dăm pop la elem. minim din heap.
Când se termină un task, procesul acestuia devine disponibil și îl vom readăuga în heap.

Pentru managementul evenimentelor specifice taskurilor, putem să mai avem un heap pentru timp.

Astfel, vom avea 2 heapuri de tip min-heap:

1) heap 1 - pentru procesare liberă:
stăm totuși indicii pro. disponibile (de vom ține pe toate la întârziere)

2) heap 2 - pentru evenimente

Fiecare nod va reține o structură de tip (timp, tip eveniment, idc),
unde tip eveniment poate fi:

- 0: terminare task $\Rightarrow$ vom readăuga indicele procesului în heap 1 la mom. potrivit (timp)
- 1: începere task - găsim procesor pt. task (pop pt. heap 1).

Prioritatea în al doilea heap e determinată în funcție de timp.

Dacă avem 2 evenimente în ac. timp, e prioritară terminarea taskului.

Pentru a face parcurgerea propriu-zisă;
initializăm heap 2 cu toate evenimentele de tip *insert task*
(folosim timpul $3i$).

Stăm la dif. momente de timp când pop. elem. din heap 2.
Ală, menținem: când facem pop. din primul heap (*insert task*),
vom adăuga evenimentul $(\text{time} + D_i, Q, \text{adre})$ în heap 2.

Conținut heap 1: $O(N)$

Intare + adăugare în heap 2: $O(M \log M)$.

Fiecare extragere / inserare a *apținuticii*, avem M evenimente de start
+ M evenimente de stop

$$\Rightarrow O(M \log M + \log N)$$

$$\Rightarrow O(M \log M + M \log N) - \text{complexitate timp}$$

$$O(M + N) - \text{complexitate spațiu (vector + cele 2 heapuri)}$$

= PROBLEMA 5 =

Putem observa:

- 1) pentru că numirea este unică, putem să avem o variabilă "scumpire" care să ~~țină~~ țină înnumărarea tuturor scumpirilor ~~de~~ până la momentul respectiv
- 2) pentru că avem acea variabilă, înțelegem modificările pe care le vom face asupra elem. din vectorul original vor fi cele de schimb.

Altfel, vom proceda în următorul mod:

calculăm suma originală + frecvențele elementelor (ca să știm ce schimbăm, pentru că nu contează poz. în vector pt. op. de schimb, doar câte schimbăm),

Analizăm o casă de tip SCHIMB $x \times y$

se întâmplă la fel și pt. y

pentru că nu am aplicat numirea, valoarea calculată este de fapt x -scumpire.

Dacă avem valori x -scumpire în vectorul necumpit, atunci suma de laia se modifică așa:

$$\text{sum} = \text{sum} - f[x\text{-scumpire}] \cdot (x\text{-scumpire}) + f[x\text{-scumpire}] \cdot (y\text{-scumpire})$$

și actualizăm frecvențele ~~$f[x\text{-scumpire}] + = f[x\text{-scumpire}]$~~ ~~$f[y\text{-scumpire}] + = f[x\text{-scumpire}]$~~

Când calculăm suma la finalul procesului par, vom afla suma la care am adăugat n scumpire



Am folosit unordered-map pentru frecvente

Avem $O(N)$ de la parcurgerea vectorului

Fiecare query se rezolvă în $O(1)$

$\Rightarrow O(N+Q)$ - complexitate timp.

$O(N)$ - complexitate spațiu (vector + unordered-map),

= PROBLEMA 6 =

Vom rezuma problema punct - o abordare off-line.

Mai întâi, vom determina pentru fiecare operație de add poziția finală pe care o va avea elementul în vectorul final. După ce avem aceste informații, vom parcurgem query-urile din nou și determinăm numele pe parcurs, înscind elementul direct în poziția corespunzătoare din vectorul final.

Pentru a determina pozițiile finale, vom folosi un arbori binari de căutare, unde fiecare nod reține (id-ul operației, dimensiunea subarborului) pentru a găsi poziția în timp logaritmic.

În acest stadiu, doar luăm în considerare operațiile de tip 1 și 2 în le săgem în arbore.

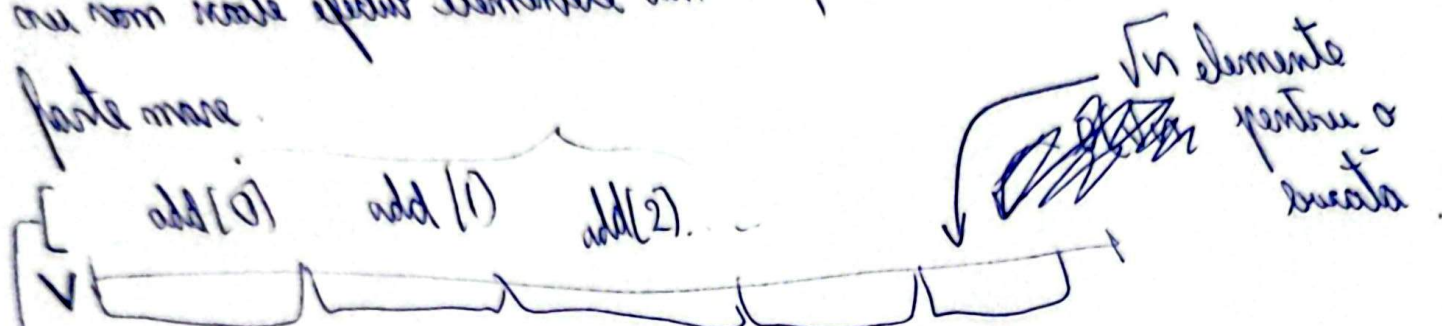
Pentru a merge vectorul final, vom parcurge arborul în ordine (dici pozițiile vor fi corectate) și vom asocia operațiilor de inserare pozițiile finale.

Parcurgem din nou query-urile, dar de data asta vom calcula numele folosind un AINT. Vom mai avea 2 variabile pentru lastred + lastblack. Când întâlnim o operație de tip 1 sau 2, vom update aint-ul înscind elem. direct în poziția lui finală, iar ~~la~~ la operațiile de tip 3, vom calcula x pe un query din nou de nume ~~la~~ folosind aint pe interval lastred - lastblack.

Arbori binari de căutare + update / query pe aint $\Rightarrow O(Q \cdot \log Q)$ - complexitate
 $O(Q)$ - complexitate spațiu (pentru arbori, query-uri, pr. asociate)

- PROBLEMA 7 =

Facem split decomposition, având în vedere faptul că la diminuare, nu vom scade efectiv elementele din vector, ci le vom înlocui cu o valoare foarte mare.



→ fiecare bucata va tine minte minimul

→ vector de $\sqrt{N}$ elemente care retine ce cat ~~este~~ ar fi trebuit să se modifice toate valorile din acel interval

Altfel minimul pentru fiecare bucata, deci pentru a găsi minimul global, doar parcurgem valorile bucatilor $\Rightarrow O(\sqrt{N})$.

Când "restem" minimul, recalculăm minimul pentru acea bucata $O(\sqrt{N})$.

!! mereu când considerăm un număr care să fie minim pe bucata, trebuie să adăugăm valoarea curentă din vectorul add (ca să facem suma elementelor noastre).

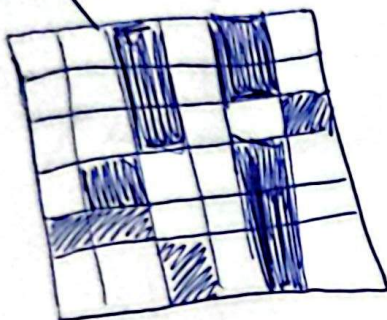
La calcularea cu Z bucata, trecem prin primele bucata până la ~~cel~~ minim și dăm update la add (nu trebuie să recalculăm minimul, tot acela e să fie pentru că calculul este uniform) + minim manual elemente din bucata incompletă + recalculăm minimul pe acel interval

$O(N\sqrt{N})$ - complexitate timp; $O(N + \sqrt{N}) = O(N)$ complexitate spațiu.

= PROBLEMA 8 =

Mai întâi, vom identifica toate "lacurile" / regiunile separate de apă de pe harta folosind flood fill, având o matrice separată care maparea fiecare pătrățel de apă cu regiunea asociată.

Exemplu:



1	1		2		3
1	1		2		3
1	1		2	2	
1		2	2		4
		2	2		4
5	5		2		4

Într-un flood fill ~~recursiv~~ (care e doar pe linii și coloane), vom sta în $O(1)$ din ce regiune sunt (x_1, y_1) și (x_2, y_2) .

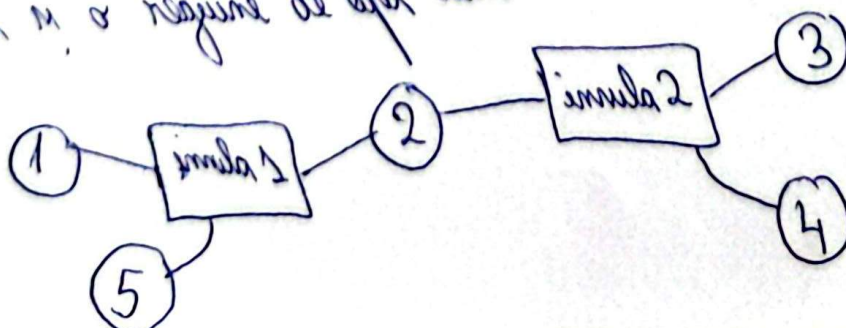
Pentru a identifica regiunile de apă adiacente cu o anumită insulă, facem flood fill pe insulă (n. diagonală, de data asta) ca să identificăm vecinii pătrățelilor de insulă.

Pă, exemplul de mai sus:

insula 1 - regiunile 1, 2, 5

insula 2 - regiunile 2, 3, 4.

Vom reprezenta harta și toate aceste regiuni sub forma unui graf: fiecare insulă + regiune de apă reprezintă un nod, avem muchii între o insulă și o regiune de apă dacă acestea sunt vecine



Altfel, putem afla numărul minim de muchii prin care trebuie să trecem ca să ajungem de la poziția (x_1, y_1) la (x_2, y_2) găsiind drumul minim dintre perechiile de așteptare acestora pe graf.

Pentru a găsi drumul minim: observăm că graful nu poate avea cicluri
 $\Rightarrow$ putem aplica algoritmi de găsirea drumului minim pe arbori.
(de exemplu, LCA).

Totuși, trebuie să luăm în considerare și faptul că mereu vom trece prin noduri de înălțare, deci:

$$\text{distanța căutată} = \text{nr. muchii pe drum minim} / 2.$$

Build full + construire graf: $O(N \cdot M)$.

Pe fiecare query, primim răspunsul în $O(\log(\text{nr. noduri})) \Rightarrow O(Q \cdot \log(N \cdot M))$

$\Rightarrow O(N \cdot M + Q \cdot \log(N \cdot M))$ - complexitate timp

$O(N \cdot M)$ - complexitate spațiu